МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**



Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Лабораторная работа по дисциплине: “ Спецификация, архитектура и проектирование программных систем”

Выполнил: ст. группы ПВ-233
Резник Богдан Витальевич

Белгород, 2026 г.

Цель работы

Получить практические навыки анализа требований к программному обеспечению, проектирования архитектуры информационной системы, формирования команды разработки и распределения задач между участниками проекта.

Задание 1. Описание проекта

Название проекта: FleetManager — веб-приложение для управления логистикой парка транспортных средств предприятия.

Назначение системы: автоматизация диспетчеризации, учёта и мониторинга автопарка предприятия: от ведения реестра машин и водителей до распределения заявок и контроля движения транспорта на карте в реальном времени.

Целевая аудитория:

- диспетчеры и логисты малых и средних предприятий (офис, ПК);
- администраторы / руководители автопарка (офис, ПК);
- водители (смартфон, мобильная версия PWA).

Основные проблемы, которые решает система:

- учёт заявок и транспорта в Excel и по телефону → потеря заявок, простои машин;
- диспетчер не видит, где находятся машины и чем они заняты;
- потеря контроля над транспортом в зонах с нестабильным GPS-сигналом — критично для Белгородской области, где наблюдаются постоянные перебои с точностью навигации;
- бесконтрольные расходы на ГСМ и пропущенные сроки ТО.

Основные функции системы:

- реестр транспортных средств и водителей;
- создание заявок на перевозку и назначение ТС/водителя (вручную и с подсказкой системы по грузоподъёмности и статусу);
- построение маршрутов через API Яндекс.Карт с учётом пробок;
- приём GPS-координат со смартфона водителя через PWA (без покупки трекеров);
- мониторинг машин на карте в реальном времени (WebSocket);
- **прогнозное позиционирование без GPS**: расчёт примерного положения по маршруту, времени в пути и загруженности дорог;
- учёт ГСМ, ремонтов, напоминания о ТО и страховке;

- аналитический дашборд (загрузка парка, простои, расходы).

Предполагаемые условия эксплуатации:

- диспетчер и администратор — стационарный ПК в офисе, браузер Chrome/Firefox, стабильный интернет;
- водитель — смартфон Android/iOS, мобильный интернет 3G/4G с возможными перебоями, работа в условиях нестабильного GPS (Белгородская область);
- сервер — Linux, Docker, облачный VPS или локальный сервер предприятия.

Задание 2. Формирование требований

2.1. Функциональные требования

ID	Название	Описание	Пользователь	Ожидаемый результат
FR-01	Аутентификация и роли	Вход по логину/паролю, JWT-токен, роли: админ, диспетчер, водитель	Все	Доступ только к функциям своей роли
FR-02	Реестр ТС	CRUD паспорта машины: марка, гос. номер, VIN, тип, грузоподъёмность, статус	Администратор	Актуальный реестр парка
FR-03	Реестр водителей	CRUD данных водителя, категории прав, статус (на смене / отпуск / больничный)	Администратор	Актуальный список водителей
FR-04	Создание заявки	Заявка: адреса подачи/доставки, груз, вес, объём, сроки	Диспетчер	Заявка в статусе «Новая»
FR-05	Назначение на заявку	Выбор ТС и водителя вручную; система предлагает свободные ТС подходящей грузоподъёмности	Диспетчер	Заявка в статусе «Назначена», водитель видит задачу
FR-06	Построение маршрута	Расчёт маршрута через API Яндекс.Карт: расстояние, время с учётом пробок	Диспетчер	Маршрут на карте, оценка ETA и расхода топлива
FR-07	Передача координат	PWA водителя запрашивает Geolocation API и	Водитель (автоматически)	Точки маршрута

ID	Название	Описание	Пользователь	Ожидаемый результат
		отправляет координаты каждые 10–30 с		сохраняются в БД
FR-08	Мониторинг на карте	Отображение всех машин на карте, обновление через WebSocket	Диспетчер	Актуальные позиции с задержкой ≤ 2 с
FR-09	Прогнозное позиционирование	При потере GPS > 60 с — расчёт позиции по маршруту, времени и загруженности дорог; лимит 30 мин, далее статус «Связь потеряна»	Система / диспетчер	Машина отображается с пометкой «Расчётная позиция»
FR-10	Коррекция позиции	При восстановлении GPS — привязка к реальной точке (snap), запись о коррекции в журнал	Система	Реальная позиция, лог расхождения
FR-11	Учёт ГСМ и ТО	Заправки (литры, сумма), ремонты, напоминания о ТО/страховке/техосмотре	Администратор, водитель	История расходов, уведомления о сроках
FR-12	Аналитика	Дашборд: загрузка парка, простои, расход топлива, стоимость км пробега	Администратор	Графики и отчёты за период
FR-13	Статусы рейса	Водитель отмечает: «Начал маршрут», «Прибыл в точку», «Выполнил»	Водитель	Диспетчер видит ход выполнения заявки

ID	Название	Описание	Пользователь	Ожидаемый результат
FR-14	Офлайн-режим PWA	Кэш задач через Service Worker, буфер координат в IndexedDB, синхронизация при появлении сети	Водитель	Данные не теряются при обрыве связи

2.2. Нефункциональные требования

Категория	Требования
Надёжность	Доступность ≥ 99% в рабочее время; буферизация координат в PWA при обрыве связи; ежедневный бэкап БД; при ошибке прогнозного алгоритма — деградация до статуса «Связь потеряна», без падения системы
Производительность	До 100 ТС одновременно; доставка координат на карту ≤ 2 с; ответ REST API ≤ 500 мс (95-й перцентиль); загрузка интерфейса ≤ 3 с; интервал отправки координат 10–30 с
Безопасность	HTTPS/WSS; пароли — bcrypt; JWT с ограниченным сроком; RBAC на уровне API; защита от SQL-инъекций (ORM); rate limiting; обработка персональных данных водителей по 152-ФЗ
Масштабируемость	Stateless-бэкенд (горизонтальное масштабирование); Redis pub/sub для WebSocket при нескольких инстансах; партиционирование таблицы gps_logs по месяцам; модульная структура с возможностью выделения Tracking в отдельный сервис
Условия эксплуатации	Браузеры Chrome/Firefox/Safari актуальных версий; Android 8+ / iOS 14+; работа при 3G и кратковременных обрывах сети; сервер: Linux, Docker, 2 CPU / 4 GB RAM минимум

Задание 3. Проектирование архитектуры

3.1. Монолитная архитектура

Основные компоненты:

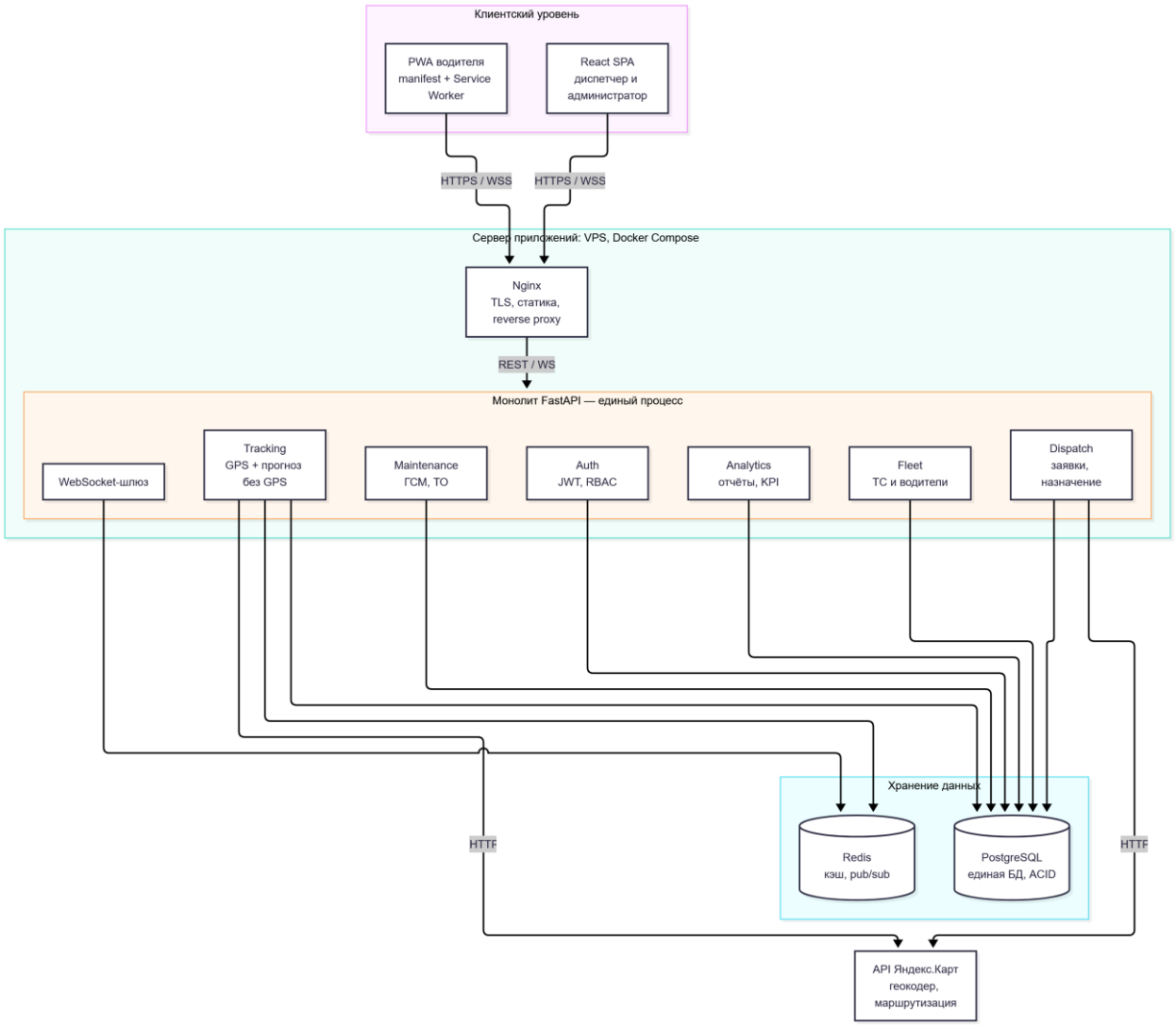
- клиент: React SPA (диспетчер/админ) + PWA (водитель);
- Nginx — раздача статики, reverse proxy, TLS;
- монолитное приложение на FastAPI с внутренними модулями: Auth, Fleet, Dispatch, Tracking (GPS + прогнозный алгоритм), Maintenance (ГСМ/ТО), Analytics;
- PostgreSQL — единая БД; Redis — кэш и pub/sub для WebSocket;
- внешний сервис: API Яндекс.Карт (геокодер, маршрутизация).

Принцип взаимодействия: клиенты ↔ сервер по REST (HTTPS) и WebSocket (WSS); модули внутри монолита общаются через прямые вызовы сервисов в рамках одного процесса; с Яндекс.Картами — HTTP-запросы с сервера.

Хранение данных: единая реляционная БД PostgreSQL (vehicles, drivers, tasks, gps_logs, fuel_logs, maintenance), миграции Alembic. Транзакции — локальные, ACID.

Развертывание: Docker Compose (nginx + app + postgres + redis) на одном VPS; CI/CD — сборка образа и деплой одной командой.

Преимущества: быстрая разработка и отладка; простое развертывание; единые транзакции; низкие требования к ресурсам и команде — оптимально для дипломного проекта. **Недостатки:** масштабируется только целиком (нагрузка от трекинга «тянет» весь сервер); изменение одного модуля требует пересборки всего приложения; при росте команды — конфликты в кодовой базе.

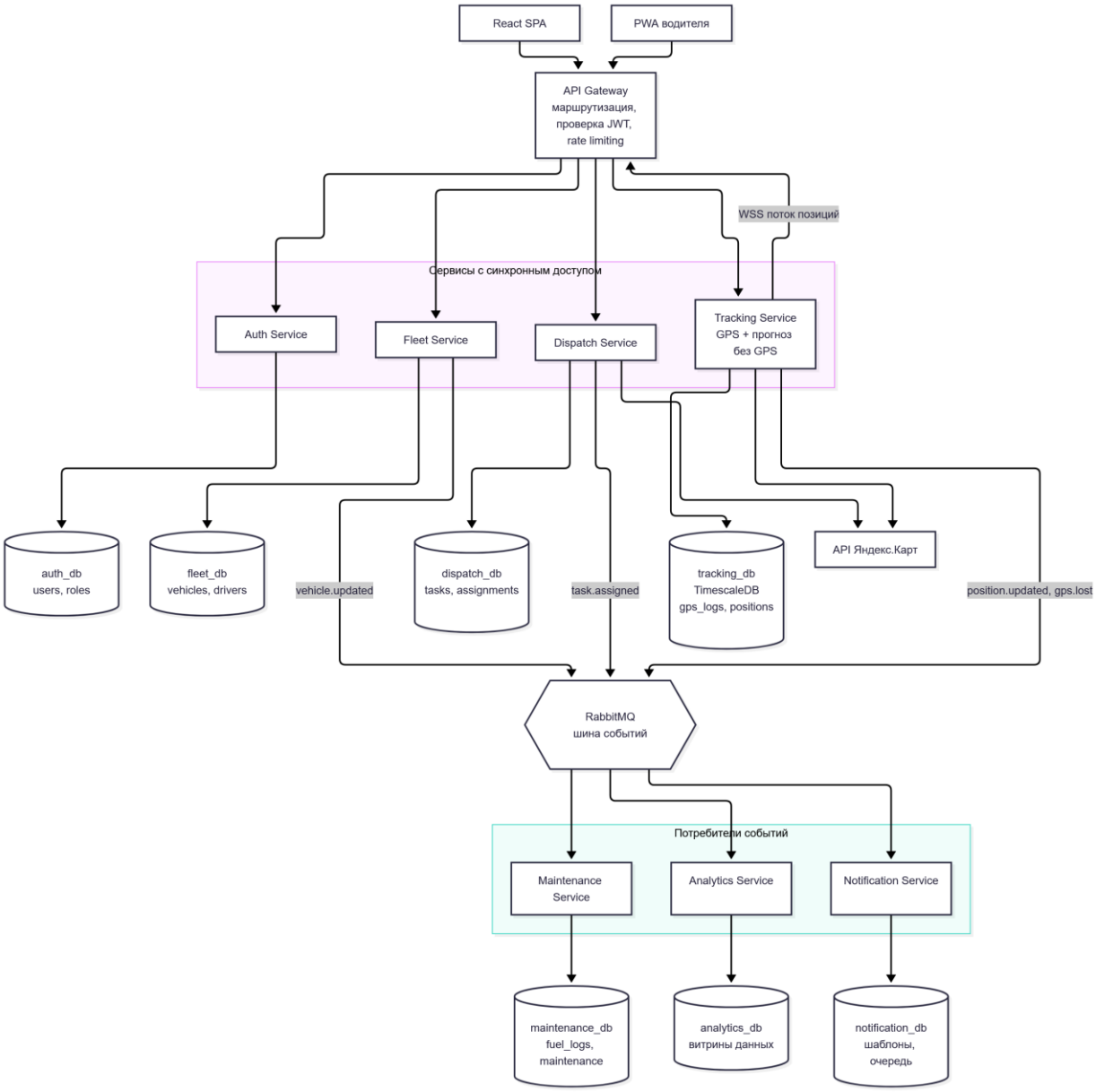

Клиентский уровень
PWA водителя
manifest + Service Worker
React SPA
диспетчер и администратор
HTTPS / WSS
HTTPS / WSS
Сервер приложений: VPS, Docker Compose
Nginx
TLS, статика, reverse proxy
REST / WS
Монолит FastAPI — единый процесс
WebSocket-шлюз
Tracking
GPS + прогноз без GPS
Maintenance
ГСМ, ТО
Auth
JWT, RBAC
Analytics
отчёты, KPI
Fleet
ТС и водители
Dispatch
заявки, назначение
Хранение данных
Redis
кэш, pub/sub
PostgreSQL
единая БД, ACID
HTTP
HTTP
API Яндекс.Карт
геокодер, маршрутизация

3.2. Микросервисная архитектура

Сервис	**Назначение**	**Основные функции**	**Данные**	**Взаимодействие**
API Gateway	Единая точка входа	Маршрутизация, проверка JWT, rate limiting	—	REST/WS с клиентами, REST/gRPC к сервисам
Auth Service	Аутентификация	Вход, выдача/обновление JWT, роли	users, roles	REST; события user.* в шину
Fleet Service	Реестр парка	CRUD ТС и водителей, статусы	vehicles, drivers	REST; события vehicle.updated
Dispatch Service	Заявки	Создание, назначение, статусы рейсов	tasks, assignments	REST к Fleet; события task.assigned
Tracking Service	Мониторинг	Приём координат, WebSocket, прогнозный алгоритм, snap-коррекция	gps_logs, positions (TimescaleDB)	События position.updated, gps.lost; WS к клиентам
Maintenance Service	ГСМ и ТО	Заправки, ремонты, напоминания	fuel_logs, maintenance	События task.completed (пробег)
Analytics Service	Отчёты	Агрегация, дашборды	витрины (read model)	Подписка на события всех сервисов
Notification Service	Уведомления	Push/email о ТО, назначенных задачах	шаблоны, очередь отправок	Подписка на события шины

Взаимодействие: синхронно — REST/gRPC через Gateway; асинхронно — шина сообщений RabbitMQ (события task.assigned, position.updated, gps.lost). У каждого сервиса своя БД (database-per-service), распределённые транзакции — через саги.

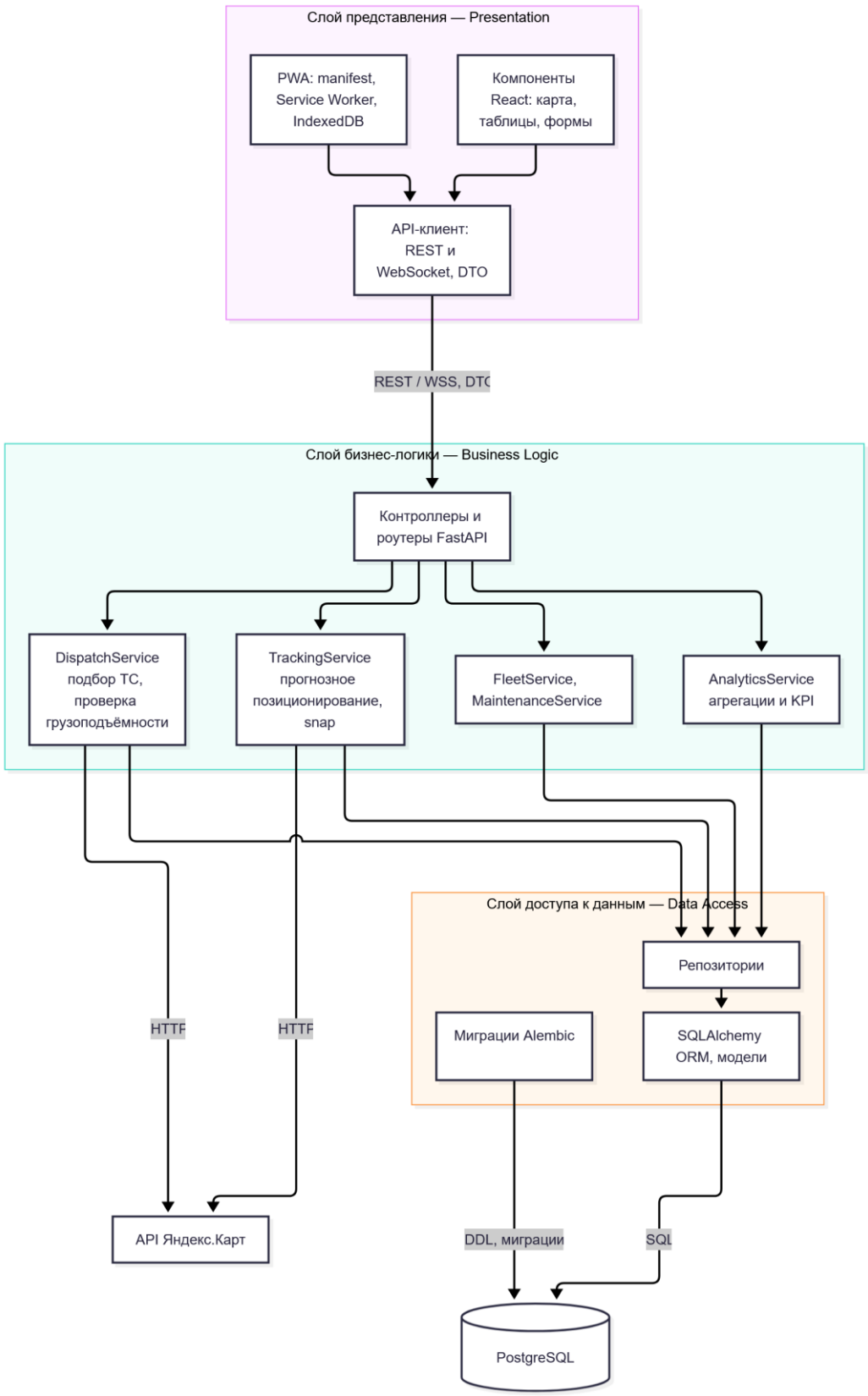
Преимущества: независимое масштабирование (Tracking — самый нагруженный); отказ одного сервиса не роняет систему; автономные команды. **Недостатки:** сложность инфраструктуры и отладки, распределённые транзакции, высокие требования к DevOps — избыточно для проекта данного масштаба.



3.3. Многослойная архитектура

Слой	Назначение	Основные компоненты	Взаимодействие
Слой представления	Интерфейс пользователя	React SPA, PWA (manifest, Service Worker), компоненты карты, формы, дашборд	Обращается к бизнес-логике через REST/WebSocket, обменивается DTO
Слой бизнес-логики	Правила предметной области	DispatchService (назначение, проверки грузоподъёмности), TrackingService (прогнозное позиционирование, snap), FleetService, MaintenanceService, AnalyticsService	Принимает DTO от контроллеров, вызывает репозитории слоя доступа к данным
Слой доступа к данным	Работа с БД	Репозитории, SQLAlchemy ORM, модели, миграции Alembic	Выполняет SQL-запросы к PostgreSQL, возвращает доменные объекты

Принцип: каждый слой обращается только к соседнему нижележащему; бизнес-логика не знает о HTTP, представление не знает о SQL. Это позволяет менять БД или UI независимо.



Вывод по заданию 3: для проекта выбран **модульный монолит с многослойной организацией внутри** — сочетает простоту развертывания монолита с чётким разделением ответственности. Микросервисная модель рассматривается как путь развития: при росте парка первым выделяется Tracking Service.

Задание 4. Техническое задание (по ГОСТ 34.602-89)

1. Введение

Система: FleetManager — веб-приложение для управления логистикой парка транспортных средств предприятия. Назначение: автоматизация диспетчеризации, учёта ТС, водителей, ГСМ/ТО и мониторинга транспорта на карте, включая расчёт позиции при отсутствии GPS-сигнала.

2. Основания для разработки

- учебное задание кафедры программного обеспечения вычислительной техники и автоматизированных систем БГТУ им. В.Г. Шухова по дисциплине «Архитектура, проектирование и ППС»;
- требования условного заказчика — транспортного предприятия, эксплуатирующего автопарк в Белгородской области в условиях нестабильного GPS-сигнала.

3. Назначение и цели создания системы

Назначение: рабочее место диспетчера, администратора и водителя в единой информационной системе. **Цели:**

- сократить простои транспорта за счёт централизованного распределения заявок (целевой ориентир — 15–20%);
- снизить расходы на ГСМ и ТО за счёт учёта и напоминаний (10–15%);
- сохранить контроль над парком при потере GPS: расчётная позиция отображается в течение до 30 минут после потери сигнала;
- заменить разрозненные Excel-таблицы и телефонные согласования единым инструментом.

4. Требования к системе

- **Функциональные:** FR-01…FR-14 (таблица задания 2.1): аутентификация и роли, реестры ТС и водителей, заявки и назначение, маршруты, приём координат с PWA, мониторинг в реальном времени, прогнозное позиционирование и коррекция, учёт ГСМ/ТО, аналитика, статусы рейса, офлайн-режим.
- **Надёжность:** доступность ≥ 99%; буферизация координат при обрывах; ежедневное резервное копирование; корректная деградация функциональности мониторинга.

- **Производительность:** 100 ТС одновременно; задержка отображения ≤ 2 с; API ≤ 500 мс.
- **Безопасность:** HTTPS/WSS, JWT, bcrypt, RBAC, rate limiting, соответствие 152-ФЗ.
- **Условия эксплуатации:** современные браузеры, Android 8+/iOS 14+, сеть 3G/4G с перебоями, сервер Linux/Docker.

5. Состав и содержание работ

Этап	Работы	Результат
1. Анализ и проектирование	Требования, ТЗ, ER-модель, схемы архитектуры, макеты UI	ТЗ, проект БД, прототипы экранов
2. Базовая инфраструктура	Репозиторий, Docker Compose, CI, миграции	Развёрнутое окружение
3. Backend: ядро	Auth, Fleet, Dispatch (REST API)	Работающие модули + Swagger
4. Frontend: ядро	SPA диспетчера/админа, реестры, заявки	Интерфейсы по макетам
5. Модуль трекинга	WebSocket, приём координат, PWA водителя, прогнозный алгоритм, snap-коррекция	Мониторинг на карте, в т.ч. без GPS
6. Учёт и аналитика	ГСМ, ТО, напоминания, дашборд	Отчёты и уведомления
7. Тестирование	Unit/integration, нагрузочный тест, полевой тест потери GPS	Протоколы испытаний
8. Внедрение и документация	Деплой, руководства, пояснительная записка	Работающая система + документы

6. Порядок контроля и приемки

- проверка каждого функционального требования по тест-кейсу (FR-01…FR-14);
- нагрузочный тест: эмуляция 100 ТС, интервал координат 10 с, критерий — задержка ≤ 2 с;
- сценарный тест «слепая зона»: эмуляция потери GPS на 10 минут → отображение расчётной позиции → восстановление сигнала → коррекция с записью в журнал;
- демонстрация заказчику сквозного сценария: заявка → назначение → рейс с мониторингом → завершение → отчёт;
- критерий приёмки: 100% FR выполнены, критические дефекты отсутствуют.

7. Требования к составу и содержанию документации

ТЗ; пояснительная записка; ER-диаграмма и описание схемы БД; спецификация API (OpenAPI/Swagger); руководство диспетчера/администратора; руководство водителя (PWA); инструкция по развертыванию; программа и методика испытаний.

8. Источники разработки

ГОСТ 34.602-89; документация FastAPI, React, PostgreSQL, Service Worker/PWA (MDN); документация API Яндекс.Карт; материалы по алгоритмам Map Matching и Dead Reckoning; лекционные материалы дисциплины.

Задание 5. Формирование команды разработки

Состав команды соответствует выбранной архитектуре (модульный монолит): **5 человек, 7 ролей** с совмещением. Для микросервисного варианта команда выросла бы до 8–10 человек (отдельные владельцы сервисов, полноценный DevOps).

Участник	**Роль(и)**	**Основные обязанности**	**Задачи**	**Результат работы**
1	Руководитель проекта + системный аналитик	Требования, ТЗ, планирование, коммуникация с заказчиком	ТЗ, backlog, приоритеты, приёмка этапов	ТЗ, план проекта, доска задач
2	Backend-разработчик	Серверная логика, API, WebSocket	Модули Auth/Fleet/Dispatch/Tracking, прогнозный алгоритм	Работающий API + Swagger
3	Frontend-разработчик	Клиентские интерфейсы	SPA диспетчера, PWA водителя, карта, Service Worker	Интерфейсы всех ролей
4	Разработчик БД + DevOps (совмещение)	Схема данных и инфраструктура	ER-модель, миграции, партиционирование gps_logs, Docker, CI/CD, бэкапы	Схема БД, стенд, пайплайн деплоя
5	Тестировщик	Качество	Тест-кейсы по FR, нагрузочный тест, сценарий «потеря GPS», регресс	Протоколы испытаний, баг-репорты

Задание 6. Доска задач (Kanban)

Инструмент: GitHub Projects. Колонки: Backlog → To Do → In Progress → Code Review → Testing → Done. Сформирована 24 задача.

ID	Задача	Исполнитель	Статус
T-01	Репозиторий, линтеры, структура проекта	Backend	To Do
T-02	Docker Compose (nginx, app, postgres, redis)	DevOps	To Do
T-03	ER-модель и миграции основных таблиц	Разработчик БД	To Do
T-04	Аутентификация JWT + bcrypt	Backend	To Do
T-05	RBAC: роли админ/диспетчер/водитель	Backend	To Do
T-06	CRUD реестра ТС (API + UI)	Backend + Frontend	To Do
T-07	CRUD реестра водителей (API + UI)	Backend + Frontend	To Do
T-08	Создание заявки: API + форма	Backend + Frontend	To Do
T-09	Назначение ТС/водителя, проверка грузоподъёмности	Backend	To Do
T-10	WebSocket-шлюз: приём и трансляция координат	Backend	To Do
T-11	PWA: manifest + Service Worker, установка на экран	Frontend	To Do
T-12	Geolocation API: отправка координат с телефона	Frontend	To Do

ID	Задача	Исполнитель	Статус
T-13	Интеграция API Яндекс.Карт: маршрут, ETA	Backend	To Do
T-14	Карта диспетчера: маркеры, статусы машин	Frontend	To Do
T-15	Алгоритм прогнозного позиционирования без GPS	Backend	To Do
T-16	Snap-коррекция при восстановлении GPS + журнал	Backend	To Do
T-17	Мобильный UI водителя: задачи, статусы рейса	Frontend	To Do
T-18	Учёт заправок и ремонтов (API + UI)	Backend + Frontend	To Do
T-19	Офлайн-буфер координат (IndexedDB) + синхронизация	Frontend	To Do
T-20	Напоминания о ТО/страховке, push-уведомления	Backend	To Do
T-21	Аналитический дашборд: графики, отчёты	Frontend + Backend	To Do
T-22	Партиционирование gps_logs, индексы	Разработчик БД	To Do
T-23	Нагрузочный тест: 100 ТС + сценарий потери GPS	Тестировщик	To Do
T-24	Руководства пользователя, деплой на прод	Все	To Do

Заключение

В ходе лабораторной работы выполнен анализ предметной области управления автопарком предприятия и сформированы 14 функциональных и 5 групп нефункциональных требований к системе FleetManager. Рассмотрены три архитектурные модели: монолитная, микросервисная и многослойная; для реализации обоснованно выбран модульный монолит с многослойной организацией, а микросервисная модель определена как направление масштабирования (с первым выделением Tracking-сервиса). Составлено техническое задание по структуре ГОСТ 34.602-89, сформирована команда из 5 человек с распределением 7 ролей и организована Kanban-доска из 24 задач. Ключевая особенность проекта — модуль прогнозного позиционирования транспорта без GPS-сигнала, обеспечивающий сохранение контроля над парком в условиях перебоев навигации, характерных для Белгородской области. Получены практические навыки анализа требований, проектирования архитектуры и организации командной разработки.